

Clearspace Software Engineering Enhancement Narrative

Payton Castle

CS 499 Milestone Two

September 15, 2026

Artifact Background

For this milestone, I enhanced Clearspace by separating responsibilities, improving file-operation feedback, and adding automated tests. I created Clearspace in August 2026 as a Windows file-management application built with C# and Windows Presentation Foundation. It supports folder navigation, file searches, tags, and operations such as copying, moving, renaming, and deleting files. This enhancement focuses on software engineering and design while keeping the application's existing features.

Reason for Selection and Improvements

I selected Clearspace for my ePortfolio because it provides a practical example of maintaining software with several connected features. Its interface, background searches, and Windows file operations create opportunities to demonstrate design decisions beyond adding a new button or screen. The original project already separated models, commands, and services, but MainViewModel coordinated too many responsibilities. I moved search coordination into SearchCoordinator, navigation loading into NavigationCoordinator, and sidebar state into SidebarViewModel. These components make individual behaviors easier to inspect and test without constructing the entire interface.

I also replaced the true-or-false results from copy, move, rename, and delete with FileOperationResult. The new result distinguishes success, cancellation, and failure, while retaining Windows diagnostic information and a message for the user. For example, canceling a

copy operation should not be presented as a permission error. Updating the callers was an important part of this change because collecting better information would have little value if the interface ignored it.

The enhanced artifact includes 87 automated tests that passed in the Release configuration. These tests check search cancellation and duplicate removal, tag persistence and assignment behavior, indexing edge cases, and file-operation outcomes. Controlled test data makes it possible to simulate failures without deleting or moving personal files. The tests provide repeatable evidence that these specific behaviors work, although they do not prove that every interface interaction is correct.

Course Outcomes and Enhancement Scope

The completed changes meet the software engineering goals described in my code review: smaller components, more informative operation results, and automated tests. They provide evidence for the outcome concerning well-founded computing techniques and tools by applying separation of responsibilities and repeatable testing to an existing application. The review, implementation notes, and this narrative also support professional technical communication by explaining the reasons for the changes and their limitations.

Input validation and explicit handling of cancellation and errors support progress toward a security mindset. These improvements help avoid misleading feedback when files are involved, but they do not guarantee that a partially completed Windows operation can be undone. My enhancement scope remains focused on software engineering. Changes to traversal depth, folder-size calculations, and SQLite tag storage remain planned for the later algorithms and database categories.

Reflection on Learning and Challenges

This work reinforced the importance of defining what each component owns before moving code. Search was especially challenging because several sources can return results at different times. Canceling an old search is not sufficient unless late results are also prevented from replacing a newer search. Separating the coordinator from its sources made these situations easier to reproduce with controlled tests.

Testing also exposed assumptions that were easy to miss during a code review. A filter-only search such as `ext:txt` could skip subfolder results because the application treated index coverage as proof that the index could answer the query. Clearing a search also restored the folder's items without restoring its status message. Both issues were corrected and covered by regression tests. This experience helped me understand why successful compilation is only one part of verification. Manual checks remain necessary for Windows dialogs and visual behavior, and the latest search fixes have not yet been rechecked in the interface.